

아주 쉬운 GpuCluster 교과서

코딩을 한 번도 안 해본 친구를 위한
GPU 나눠 쓰기 이야기

12개 챕터 · 그림과 비유로 배우는 입문 트랙



아주 쉬운 GpuCluster 교과서

안녕하세요! 이 책은 **GPU를 여러 명이 나눠 쓰는 신기한 프로그램**에 대한 이야기예요.

코딩을 한 번도 안 해봤어도 괜찮아요. 컴퓨터를 잘 몰라도 괜찮아요.

저랑 같이 **그림과 비유**로 천천히 알아가 봐요. 😊



이 책은 누구를 위한 걸까요?

- 컴퓨터가 어떻게 일하는지 궁금한 친구
- "GPU"라는 말을 들어봤지만 뭔지 모르는 친구
- 어려운 책(어른용)을 보고 "으악 너무 어려워!" 했던 친구

이 책은 **어려운 말을 안 써요.**

꼭 필요한 어려운 말이 나오면, 먼저 **우리가 아는 것**(급식, 사물함, 운동장 같은 것)에 빗대서 설명할 거예요.



목차 (읽는 순서)

1. 컴퓨터와 두뇌, 그리고 GPU

컴퓨터 안에는 "두뇌"가 있어요. GPU는 그림을 아주 잘 그리는 특별한 두뇌예요.

2. GPU를 왜 나눠 쓸까요?

GPU는 크고 비싼데, 한 명이 다 못 써요. 그래서 나눠 쓰면 좋아요.

3. 프로그램이 GPU에게 일을 시켜요

프로그램은 GPU에게 "자리 좀 줘!"라고 주문해요. 식당 주문서처럼요.

4. 가로채기 마법! (후킹)

이 책의 가장 중요한 부분! 주문서를 중간에서 몰래 가로채서 검사해요.

5. 메모리 칸 나눠 주기 (quota)

각자 정해진 양만 쓰게 해요. 급식 정량처럼요.

6. 각자의 방, 컨테이너

안 싸우게 각자 칸막이 방을 줘요. 독서실처럼요.

7. 접속 창구, 서버

"GPU 방 주세요!" 하면 만들어 주는 친절한 접속처가 있어요.

8. 전체 설계도

지금까지 배운 부품들이 어떻게 하나의 서비스가 되는지 큰 그림으로 봐요.

9. 신청 한 건이 흘러가는 길

버튼 하나 누르면 어떤 일이 일어나는지 처음부터 끝까지 따라가 봐요.

10. 같이 쓰면 빨라질까? 우리 실험

진짜로 실험해 봤어요! 결과가 재미있어요.

11. 아직 못 하는 것 (정직한 한계)

이 프로그램이 못 하는 것도 솔직하게 알려줄게요.



이 책을 읽는 법

- **순서대로** 읽어 주세요. 뒤 이야기는 앞 이야기를 알아야 이해돼요.
- 챕터 끝에는  **한 줄 정리**가 있어요. 그것만 기억해도 충분해요.
- **? 스스로 물어보기**도 있어요. 답이 같이 있으니 먼저 혼자 생각해 보고 확인해 보세요.
- 그림(네모랑 화살표)이 나오면, 천천히 손가락으로 따라가 보세요.
네모는 부품, 화살표는 정보가 흘러가는 방향이에요.

어려워 보이죠? 사실 별거 아니에요. 우리 같이 가요! 



첫 장으로: 01. 컴퓨터와 두뇌, 그리고 GPU

1장. 컴퓨터와 두뇌, 그리고 GPU 🧠

이 장에서 배울 것: 컴퓨터 안에는 "두뇌"가 있어요. 그리고 GPU는 아주 특별한 두뇌예요.

🎯 오늘의 목표

1. 컴퓨터에도 "생각하는 부분"이 있다는 걸 알기
2. CPU랑 GPU가 뭐가 다른지 알기
3. 왜 GPU가 어떤 일은 훨씬 빠르지 알기

컴퓨터 안에는 작은 두뇌가 있어요

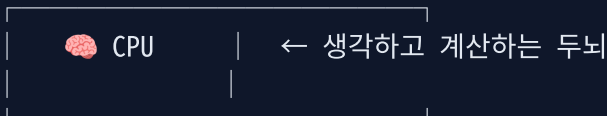
사람은 머리로 생각하죠?

컴퓨터도 똑같아요. 컴퓨터 안에는 **생각하고 계산하는 부분**이 있어요.

그 부분을 **CPU**라고 불러요.

(CPU = 컴퓨터의 똑똑한 두뇌, 라고 기억하면 돼요.)

컴퓨터



CPU는 아주 똑똑해요. 어려운 문제도 척척 풀어요.

하지만 CPU는 일꾼이 **몇 명밖에 없어서** 한 번에 한 가지씩 순서대로 풀어요.

아주 똑똑한 선생님 몇 분이 문제를 푸는 것 같아요.

GPU는 그림을 잘 그리는 특별한 두뇌예요

컴퓨터에는 두뇌가 하나 더 있을 수 있어요.

바로 **GPU**예요.

GPU는 원래 **그림과 영상을 빠르게 그리려고** 만든 두뇌예요.
게임 화면이나 영상이 부드럽게 움직이는 건 GPU 덕분이에요.

컴퓨터

 CPU	 GPU
똑똑한 한명	그림 잘그리는 일꾼 아주많은

그런데 재미있는 점!

GPU 안에는 **작은 일꾼이 아주아주 많이** 들어 있어요.

한 명 한 명은 그렇게 똑똑하지 않아요.

하지만 **숫자가 엄청 많아서**, 비슷한 일을 한꺼번에 하면 정말 빨라요.

한 명이 1000문제 vs 1000명이 1문제씩

비유로 생각해 봐요.

산수 문제가 **1000개** 있어요. 다 비슷한 더하기 문제예요.

CPU 방식 — 똑똑한 선생님 몇 명이 한 줄로 서서 순서대로 풀어요.

1번, 2번, 3번... 한 문제씩 차례로. 1000번 반복해야 해요.

CPU (선생님 몇 명):

[①] [②] [③] [④] [⑤] ... (한 번에 몇 문제씩, 순서대로)

GPU 방식 — 친구 1000명이 동시에 한 명씩 한 문제만 풀어요.

다 같이 동시에 풀어요. 그러면 한 번에 끝나요!

GPU (친구 1000명, 동시에!):

[①][②][③][④][⑤][⑥][⑦][⑧][⑨][⑩]... (뽕뽕이 동시에)

[⑪][⑫][⑬][⑭][⑮][⑯][⑰][⑱][⑲][⑳]...

... (한 번에 끝!)

비슷한 문제가 아주 많을 때는 **GPU가 훨씬 빨라요.**

그래서 요즘 인공지능(AI)을 가르칠 때 GPU를 많이 써요.

AI 공부는 비슷한 계산을 **엄청나게 많이** 하거든요.

예를 들어 사진 속 고양이를 알아맞히거나, 챗봇이 글을 쓰는 것 같은 일이에요.

(사실은 GPU도 아무 일이나 다 빠른 건 아니에요. *비슷한 일이 아주 많을 때만* 빨라요. 지금은 이렇게 알면 충분해요.)

한 줄 정리

CPU는 똑똑한 한 명, GPU는 평범하지만 **아주 많은 일꾼**.

비슷한 일이 많을 때는 일꾼이 많은 GPU가 빨라요.

? 스스로 물어보기

Q1. CPU랑 GPU 중에 "일꾼이 많은" 쪽은 누구일까요?

👉 GPU예요. 작은 일꾼이 아주 많아요.

Q2. 비슷한 문제 1000개를 풀 때 GPU가 빠른 이유는?

👉 일꾼이 많아서 **한꺼번에** 풀 수 있으니까요.

Q3. 똑똑한 한 명이 어려운 문제 하나를 푸는 건 누가 더 잘할까요?

👉 CPU예요. 한 가지 어려운 일은 똑똑한 한 명이 나아요.

➡ 다음 장: GPU는 크고 비싼데, 왜 여러 명이 나눠 쓸까요?

02. GPU를 왜 나눠 쓸까요?

2장. GPU를 왜 나눠 쓸까요? 🍰

이 장에서 배울 것: GPU는 크고 비싼데, 한 명이 다 못 써요. 그래서 나눠 쓰면 좋아요.

🎯 오늘의 목표

1. GPU가 왜 "아깝게" 쓰일 수 있는지 알기
2. 나눠 쓰면 왜 좋은지 알기
3. "분수처럼 쪼개기"가 무슨 말인지 알기

GPU는 크고 비싸요

지난 장에서 GPU는 일꾼이 아주 많은 두뇌라고 했죠?

그런데 이 GPU는 **아주 비싸요**. 그리고 **아주 커요**.

GPU 안에는 **메모리**라는 게 있어요.

메모리는 일을 하려고 **물건을 잠깐 올려두는 큰 책상**이라고 생각해요.

GPU 의 책상 (메모리)

아주 넓은 책상!
(예: 12칸짜리 큰 책상)

이 책상은 아주 넓어요. 예를 들면 **12칸**짜리예요.

(진짜로는 "12 기가바이트" 정도예요. 그냥 "아주 큰 책상"이라고 알면 돼요.)

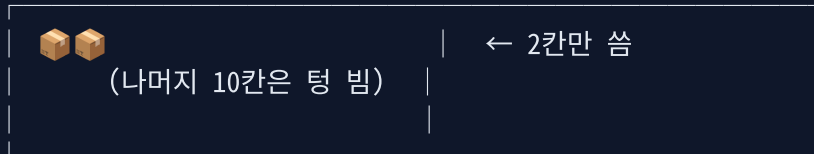
한 명이 다 못 써요

그런데 문제가 있어요.

어떤 친구는 GPU에서 **작은 일**만 해요.

12칸짜리 큰 책상에서 **2칸**만 쓰고 나머지는 비어 있어요.

친구 한 명이 쓸 때



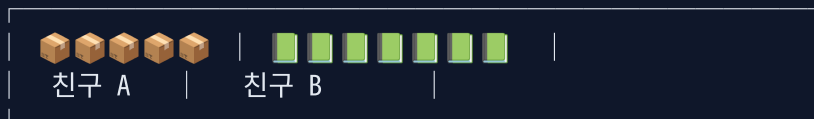
아깝다! 😞

10칸이 그냥 놀고 있어요. 너무 아깝죠?
비싼 GPU인데 거의 안 쓰고 있는 거예요.

나눠 쓰면 좋아요!

좋은 생각이 있어요.
큰 책상을 **여러 친구가 나눠 쓰면** 되잖아요!

여러 명이 나눠 쓸 때



꼭 차게 잘 씬! 😊

큰 운동장을 떠올려 봐요.
한 사람만 쓰면 운동장이 텅텅 비어요.
여러 반이 구역을 나눠 쓰면 운동장을 알차게 쓰죠!

큰 케이크도 마찬가지예요.
혼자 다 못 먹으면 친구들이랑 나눠 먹으면 돼요. 🍰

"분수처럼 쪼개기"가 뭐예요?

여기서 중요한 말이 나와요. **fractional**(프랙셔널)이에요.
어려운 말 같지만 뜻은 쉬워요. "**비율로 쪼갬다**"는 뜻이에요.

0.4, **0.6** 같은 **점이 찍힌 숫자**를 본 적 있나요? (어려운 말로 '소수'라고 해요. 2, 3, 5, 7 같은 수학 시간의 소수(素數)와는 다른 말이에요. 여기서는 그냥 "점이 찍힌 숫자"라고 기억해요.)

- 0.4 는 100칸 중 40칸 = **전체의 40%** — 큰 책상의 거의 절반보다 조금 적게
- 0.6 은 100칸 중 60칸 = **전체의 60%** — 큰 책상의 절반보다 조금 많게

12칸 책상을 0.4 / 0.6 으로 나누기

친구 A (0.4)	친구 B (0.6)
←약 5칸→	←약 7칸→

합치면 $0.4 + 0.6 = 1.0$ (전체 다 씀!)

이렇게 GPU를 분수처럼 쪼개서 나눠 주는 게 이 프로그램이 하는 일이에요.
우리 프로그램 이름이 **fGPU**인데, 앞의 **f**가 바로 이 "분수(fractional)"의 f예요!

(사실은 책상이 정확히 칸으로 나뉘진 않지만, "비율로 나눈다"는 느낌만 알면 충분해요.)

한 줄 정리

GPU는 크고 비싼데 한 명은 조금만 써요.
그러니 **분수처럼 쪼개서** 여러 명이 나눠 쓰면 알차게 쓸 수 있어요.

? 스스로 물어보기

Q1. 친구 한 명이 큰 GPU를 조금만 쓰면 뭐가 문제예요?

👉 나머지가 놀아서 아까워요.

Q2. 0.4 랑 0.6 을 더하면 얼마예요? 그건 무슨 뜻이에요?

👉 1.0 이에요. GPU 전체를 다 쓴다는 뜻이에요.

Q3. "fractional(비율로 쪼개기)"은 무슨 뜻이에요?

👉 GPU를 비율(0.4, 0.6 같은 점이 찍힌 숫자)로 나눠 쓴다는 뜻이에요. 0.4는 40%예요.

➡ 다음 장: 그런데 친구가 GPU에게 "자리 줘!"라고 어떻게 말할까요?

03. 프로그램이 GPU에게 일을 시켜요

3장. 프로그램이 GPU에게 일을 시켜요 🍴

이 장에서 배울 것: 프로그램은 GPU에게 "자리 좀 줘!"라고 주문해요. 식당 주문서처럼요.

🎯 오늘의 목표

1. "프로그램"이 뭔지 알기
2. 프로그램이 GPU에게 어떻게 부탁하는지 알기
3. "메모리 좀 줘"라는 주문이 뭔지 알기

프로그램이 뭐예요?

프로그램은 컴퓨터한테 시키는 **할 일 목록**이에요.

"이거 해, 저거 해" 하고 적어둔 **약속 목록** 같은 거예요.

예를 들어 그림을 그리는 프로그램, 게임 프로그램, AI 프로그램이 있어요.

AI 프로그램은 GPU의 도움을 많이 받아요. (1장 기억나죠? 일꾼이 많아서요.)

예: 글을 써 주는 챗봇 AI도 GPU 없이는 너무 느려요.

프로그램 (할 일 목록)

- | | |
|--------------|--|
| 1. GPU 자리 받기 | |
| 2. 계산하기 | |
| 3. 자리 돌려주기 | |

GPU에게 "메모리 좀 줘!" 주문하기

프로그램이 GPU에서 일을 하려면 **책상 자리(메모리)**가 필요해요.

(2장에서 메모리는 "물건 올려두는 큰 책상"이라고 했죠?)

그래서 프로그램은 GPU에게 이렇게 말해요.

"책상 자리 2칸만 줘!"

이 부탁을 컴퓨터 말로 `cudaMalloc` (쿠다말록)이라고 해요.

어려워 보이죠? 그냥 **"GPU야, 메모리 자리 좀 줘"**라는 주문이라고 알면 돼요.

프로그램: "cudaMalloc — GPU야, 2칸 줘!"



GPU: "그래, 여기 2칸 줄게~" 📦📦

식당 주문서처럼 생각해 봐요

식당에 가면 주문서를 써서 주문하죠?

"비빔밥 하나 주세요!"

GPU도 똑같아요. 프로그램이 **주문서**를 써서 GPU에게 줘요.

- 주문 내용: **"메모리 자리 주세요"** (이게 `cudaMalloc`)

- 다 쓰면: **"자리 돌려드릴게요"** (이건 `cudaFree` — 쿠다프리)

주문서 (`cudaMalloc`)

주문: 메모리 2칸

받는 곳: GPU



▼ (주방으로)
GPU 가 자리 마련

다 쓴 뒤 (`cudaFree`)

"자리 돌려드려요!"

이렇게 **주문하기**(`cudaMalloc`)와 **돌려주기**(`cudaFree`)를 반복하면서
프로그램이 GPU를 써요.

📦 한 줄 정리

프로그램은 GPU에게 **주문서**를 줘요.

"메모리 자리 줘"가 `cudaMalloc`, "돌려줄게"가 `cudaFree` 예요.

? 스스로 물어보기

Q1. 프로그램이 GPU에게 "자리 줘"라고 하는 주문 이름은?

👉 `cudaMalloc` 이에요.

Q2. 다 쓰고 자리를 돌려주는 주문 이름은?

👉 `cudaFree` 예요.

➡ 다음 장: 이제 진짜 마법이 나와요!

주문서가 GPU로 가는 길에 **몰래 가로채서** 검사해요.

04. 가로채기 마법! (후킹)

4장. 가로채기 마법! (후킹) ✨

이 장에서 배울 것: 주문서가 GPU로 가는 길목에서 우리가 먼저 가로채서 검사해요. 이게 이 책에서 가장 중요한 마법이에요!



오늘의 목표

1. "가로채기"가 무슨 말인지 알기
2. 컴퓨터가 어떻게 우리 도구를 먼저 읽는지 알기
3. 주문이 통과(ALLOW)되거나 거절(DENY)되는 게 어떤 상황인지 알기

택배 아저씨 이야기

상상해 봐요.

내가 온라인 쇼핑몰에서 뭔가를 주문했어요.

원래는 택배 아저씨가 창고에서 바로 물건을 가져다줘요.

그런데 어떤 날, **검사원 아주머니**가 택배 아저씨 앞에 서 계세요.

아주머니가 모든 택배 상자를 하나씩 열어봐요.

"이건 규격에 맞네, 통과!" 하면 물건이 배달돼요.

"이건 너무 크고 위험해, 안 돼!" 하면 배달이 안 돼요.

보내는 사람 -> [검사원 아주머니] -> 택배 아저씨 -> 받는 사람
 ^
 이 분이 가로채는 분!

우리 프로그램도 이 감사원 아주머니처럼 일해요.

프로그램이 GPU에게 "메모리 줘!"라고 주문을 보낼 때,

그 주문이 GPU로 가는 길에 우리가 먼저 가로채서 검사해요.

"먼저 읽기 목록" — 어떻게 가로채는 게 가능할까요?

컴퓨터는 프로그램을 켤 때 이렇게 생각해요.

우리가 그 목록 맨 앞에 우리 파일을 써 놓으면,
진짜 GPU 도구보다 우리 것이 먼저 끼어들어요.

"먼저 끼어들기 목록"에 우리 이름을 써 둔 덕분에,
가짜 점원(우리 도구)이 항상 가장 먼저 서 있을 수 있어요.

우리 가로채기 도구의 이름은 **libfgpu.so**

우리가 만든 가로채기 도구는 **libfgpu.so** (리브에프지피유점에스오)라는 파일이에요.

.so는 **혼자서는 못 켜지고, 다른 프로그램 옆에 붙어서 도와주는 부품 파일**이에요.
레고 한 조각처럼 다른 프로그램에 끼워 쓰는 거예요.

파일 이름을 풀면 이래요.

- **lib** = 도구 모음 (자주 쓰는 도구들을 모아둔 상자)
- **fgpu** = fractional GPU (비율로 나눠 쓰는 GPU)
- **.so** = 다른 프로그램에 붙어서 도와주는 부품 파일

한 줄로 말하면: **"비율로 GPU를 나눠 쓰기 위한 가로채기 도구 파일"**이에요.

[libfgpu.so 가 하는 일]

```
+-----+
| 1. 주문이 들어오면 먼저 받는다 |
| 2. "양이 괜찮나?" 검사한다     |
| 3. 괜찮으면 진짜 GPU에 넘긴다  |
| 4. 너무 많으면 거절한다        |
+-----+
```

가로채면 무슨 검사를 해요?

주문이 들어오면 **libfgpu.so**는 세 가지를 차례로 확인해요.

1단계: 지금 이 친구가 이미 쓰고 있는 양은 얼마지?
2단계: 이번에 달라는 양은 얼마지?
3단계: 둘을 더했을 때 이 친구의 몫보다 많은가?

결론은 두 가지예요.

- **안 넘어**: "통과! (ALLOW)" — 진짜 GPU에 주문을 넘겨줘요.
- **넘어**: "거절! (DENY)" — 주문을 막아요. GPU는 아무것도 모르고 있어요.

주문이 들어옴
|
v

```

1단계: 지금 쓰는 양 확인    (예: 256 MiB)
|
v
2단계: 이번에 달라는 양 확인  (예: 6144 MiB)
|
v
3단계: 256 + 6144 = 6400 MiB, 내 몫(3276 MiB)보다 많아?
|
YES (많아!)                NO (괜찮아!)
|                            |
v                            v
DENY                        ALLOW
(거절, 에러 반환)          (통과, GPU에 넘김)

```

실제 로그를 보면 이렇게 생겼어요

이 프로그램이 실제로 동작하면 화면에 이런 글자들이 나와요.
(컴퓨터 화면에 찍히는 메모를 "로그"라고 해요.)

```

[fgpu] ALLOW cudaMalloc size=256MiB  used=256/3276 MiB
[fgpu] DENY  cudaMalloc size=6144MiB  used=256/3276 MiB
[fgpu] FREE  size=256MiB              used=0/3276 MiB

```

읽는 법.

- **ALLOW** = "통과!"
- **DENY** = "거절!"
- **FREE** = "자리 돌려줌"
- **used=256/3276** = "내 몫 3276 MiB 중에 256 MiB 쓰는 중"

여기서 **3276**이 어디서 왔는지 궁금할 거예요.

MiB(메비바이트)는 MB(메가바이트)랑 거의 같은데, 컴퓨터가 세기 편한 방식이에요.

1024 MiB가 약 1GB라서, 8GB는 약 8192 MiB가 돼요.

거기에 0.4를 곱하면 **8192 × 0.4 ≈ 3276 MiB** 가 내 몫(quota)이에요.

(사실은 더 복잡하지만 지금은 "전체 크기 × 비율 = 내 몫"으로 이해하면 충분해요.)

한 줄 정리

`libfgpu.so`는 "먼저 끼어들기 목록" 덕분에 GPU 주문서를 **중간에 가로채서** 검사하고, 몫을 넘으면 DENY, 괜찮으면 ALLOW해요.

? 스스로 물어보기

Q1. 우리 가로채기 도구 파일의 이름은?

👉 `libfgpu.so` 예요.

Q2. ALLOW와 DENY는 각각 어떤 뜻이에요?

👉 ALLOW는 "통과! GPU에 주문 넘겨줘", DENY는 "거절! 너무 많이 신청했어"예요.

Q3. "먼저 끼어들기 목록"은 어떤 역할을 해요?

👉 컴퓨터가 프로그램을 켤 때 우리 도구를 맨 먼저 읽게 해줘요. 그래서 가로채기가 가능해요.

➡ 다음 장: 그럼 "몫(quota)"은 어떻게 정할까요? 정확히 얼마만큼 줘야 할까요?

05. 메모리 칸 나눠 주기 (quota)

5장. 메모리 칸 나눠 주기 (quota)

이 장에서 배울 것: 각자 정해진 양만 쓰게 해요. 사물함 칸처럼, 내 칸을 넘으면 안 돼요.

오늘의 목표

1. "quota(쿼타)"가 무슨 뜻인지 알기
2. 비율(0.4, 0.6)이 실제로 몇 칸인지 알기
3. 칸을 넘으면 어떻게 되는지 알기

학교 사물함 이야기

학교에는 사물함이 있죠?

사물함에는 **칸**이 있어요.

내 칸은 내 것이고, 친구 칸은 친구 것이예요.

내가 아무리 물건이 많아도,

내 칸을 넘쳐서 친구 칸까지 쓰면 안 돼요.

GPU 메모리도 똑같아요.

각자 정해진 칸만큼만 쓰게 해요.

이 "정해진 칸"을 **quota(쿼타)**라고 불러요.

쉬운 말로 하면 **"내 뒀"** 이에요.

GPU 메모리 사물함

친구 A 칸 (quota 0.4)	친구 B 칸 (quota 0.6)
-----------------------	-----------------------

A는 이 안에서만 B는 이 안에서만

비율이 실제로 얼마예요?

2장에서 GPU 메모리를 "12칸짜리 큰 책상"이라고 했죠?

(실제로는 8기가바이트(GB)짜리 GPU가 흔해요. 그냥 "아주 큰 책상"으로 알면 충분해요.)

0.4(비율) 이면:

전체 책상 크기 $\times 0.4 =$ **내 몫**

예를 들어 전체가 8GB이면:

$8\text{GB} \times 0.4 = 3.2\text{GB}$ ← 친구 A의 몫
 $8\text{GB} \times 0.6 = 4.8\text{GB}$ ← 친구 B의 몫

합치면 $3.2 + 4.8 = 8\text{GB}$ — GPU를 빈틈 없이 꽉 채워서 나눠요!

전체 GPU 메모리 (8GB = 100%)

친구 A (0.4)	친구 B (0.6)	
3.2GB까지	4.8GB까지	

$3.2 + 4.8 = 8$ (딱 맞게 꽉!)

급식 정량처럼 생각해요

학교 급식도 **정량**이 있죠?

밥을 한 숟가락만 더 달라고 해도 아주머니가 안 주실 때가 있잖아요.

"오늘은 이만큼씩만이야!"

그것처럼 GPU도:

- 각자 정해진 양(quota)이 있어요.
- 그 양을 넘어서 달라고 하면 → **DENY(거절)**!

급식 아주머니 (= 가로채기 도구 libfgpu.so):

"밥 3그릇 줘요!" → "안 돼! 너는 1그릇까지야 (DENY)"
"밥 반 그릇 줘요" → "그래, 여기 있어! (ALLOW)"

쓰다가 돌려주면 다시 쓸 수 있어요

사물함도 마찬가지로?

내 물건을 꺼내면 칸에 자리가 생기잖아요.

그럼 나중에 다른 물건을 넣을 수 있어요.

GPU도 똑같아요.

- 주문: "2칸 줘" → 2칸이 줄어요.

- 돌려줌: "2칸 반납" → 2칸이 다시 생겨요.

처음: [내 몫: 5칸 남아]
주문 2칸: [내 몫: 3칸 남아]
주문 2칸: [내 몫: 1칸 남아]
주문 2칸: → "칸이 부족해! DENY" ← 1칸밖에 없는데 2칸 달라고 했으니
반납 2칸: [내 몫: 3칸 남아]

합이 1.0이면 딱 맞게 나눈 거예요

$$0.4 + 0.6 = 1.0$$

이건 GPU 전체를 **빈틈 없이 딱 맞게** 나눈 거예요.

합이 1.0보다 크면 → 사실은 너무 많이 나눠준 것!

(예: $0.7 + 0.7 = 1.4$ → 전체보다 40%나 더 나눠주려 함)

이럴 때는 누군가가 미리 막아줄 수 있어요.

그 역할을 하는 것을 **입장 조절**(나중에 7장에서 자세히 배워요!)이라고 불러요.

지금은 그냥 "누군가 미리 막아줘요"로만 알면 돼요.

좋은 예: $0.4 + 0.6 = 1.0$ (딱 맞음)
나쁜 예: $0.7 + 0.7 = 1.4$ (합이 1 초과 — 접수처에서 막음)

한 줄 정리

quota는 "내 몫"이에요.

비율(0.4, 0.6)을 전체 GPU 크기에 곱하면 내 몫이 나오고,

그 몫을 넘으면 주문이 거절돼요.

? 스스로 물어보기

Q1. quota를 쉬운 말로 바꾸면?

👉 "내 몫"이에요. 내가 쓸 수 있는 최대 양이에요.

Q2. GPU가 8GB인데 ratio(비율)가 0.4면 내 몫은 몇 GB예요?

👉 $8 \times 0.4 = 3.2\text{GB}$ 예요.

Q3. 내 몫이 3.2GB인데, 4GB를 달라고 하면 어떻게 돼요?

👉 DENY! $4 > 3.2$ 이니까 거절돼요.

➡ 다음 장: quota를 따로따로 주려면, 각자 "방"이 있어야 해요.

06. 각자의 방, 컨테이너

6장. 각자의 방, 컨테이너 🏠

이 장에서 배울 것: 친구들이 GPU를 쓸 때 서로 방해 안 하도록 각자 칸막이 방을 줘요.

🎯 오늘의 목표

1. "컨테이너"가 뭔지 알기
2. 방마다 가로채기 도구가 따로 있다는 걸 알기
3. 방들이 서로 모른다는 게 왜 중요한지 알기

독서실 칸막이 이야기

공부방이나 도서관에서 **칸막이 책상**을 본 적 있나요?

한 칸 한 칸이 작은 방처럼 나뉘어 있어요.

내 칸에서 공부하면 옆 친구 소리가 덜 들리고,

내 물건이 옆 칸으로 넘어가지도 않아요.

각자 자기 칸에서 따로따로 공부할 수 있어요.

독서실 칸막이

+-----+	+-----+	+-----+
내 칸	친구칸	친구칸
공부중	공부중	공부중
+-----+	+-----+	+-----+

(서로 방해 안 함!)

컴퓨터에서 이런 "칸막이 방"을 **컨테이너(container)**라고 불러요.

컨테이너는 "여러 프로그램이 서로 방해 안 하도록 따로 나눈 공간"이에요.

방을 만들어 주는 도구 — Docker

"그럼 방은 누가 만들어요?"

방을 만들어 주는 도구 이름이 **Docker(도커)**예요.

"도커"는 영어로 항구에서 짐을 내리는 사람인데,

컴퓨터에서는 "프로그램이 살 수 있는 방을 척척 만들어 주는 도구"예요.

Docker (도커) = 방 만들어 주는 도구

[방 주세요!] -> Docker -> [여기 새 방이에요!]

```
+-----+
| 컨테이너 방 |
| 프로그램 살 |
| 수 있어요!  |
+-----+
```

도커가 방을 만들면, 그 방 안에서 프로그램이 돌아요.

방 안의 프로그램은 다른 방에 무슨 프로그램이 있는지 몰라요.

각자 "내가 이 컴퓨터 전체를 쓰는 것 같다"고 느껴요.

방마다 가로채기 도구를 넣어요

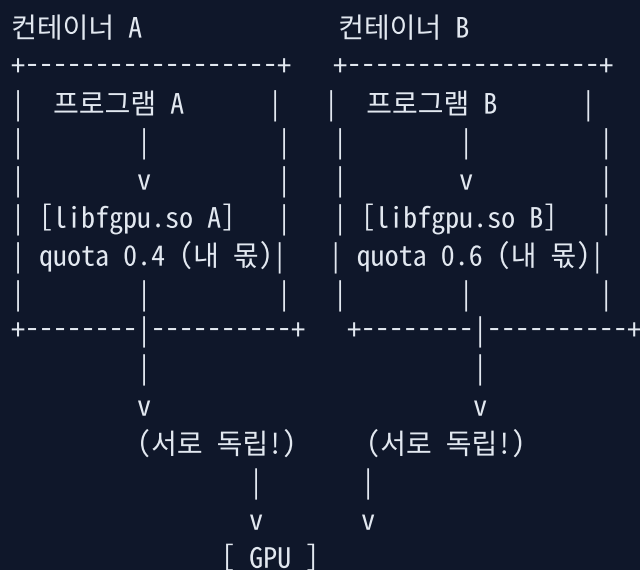
이제 중요한 부분이에요!

방이 만들어질 때, 그 방의 **"먼저 끼어들기 목록"** 맨 앞에 `libfgpu.so` 를 적어 넣어줘요.

(4장에서 배운 "먼저 끼어들기 목록" 기억하죠?)

그래서 그 방 안 프로그램의 주문은 자동으로 우리 도구를 먼저 거쳐요.

각 방에는 자기만의 가로채기 도구가 있는 셈이에요.



A의 가로채기 도구는 A의 몫(0.4)만 알아요.
B의 가로채기 도구는 B의 몫(0.6)만 알아요.
둘은 서로 무슨 주문을 하는지 전혀 몰라요!

왜 각자 따로 아는 게 좋아요?

서로 모르니까 **섞이지 않아요.**

예를 들어, 친구 A가 프로그램을 실수로 잘못 짜서 메모리를 많이 달라고 해도,
A의 가로채기 도구가 "이건 A 몫을 넘어!" 하고 막아줘요.
그 실수가 친구 B에게는 영향이 없어요.
B는 자기 몫(0.6)을 그대로 쓸 수 있어요.

친구 A가 실수를 해도:

컨테이너 A	컨테이너 B
+-----+	+-----+
DENY!	OK!
(A 몫 초과)	(영향 없음)
+-----+	+-----+
A만 에러 받음	B는 멀쩡히 일함

이게 컨테이너의 힘이에요.

각자 칸막이 방에 살아서 서로 영향을 안 줘요.

한 줄 정리

Docker가 만들어 주는 컨테이너(방)마다

`libfgpu.so`가 들어 있어서 각자 따로 quota를 지켜요.

? 스스로 물어보기

Q1. 컨테이너를 독서실에 비유하면?

👉 독서실 칸막이예요. 각자 방이 따로 있어서 서로 방해 안 해요.

Q2. 방을 만들어 주는 도구 이름은?

👉 Docker(도커)예요.

Q3. A의 `libfgpu.so` 는 B가 얼마나 쓰는지 알까요?

👉 몰라요. 각자의 도구는 자기 방의 몫만 알아요.

➡ 다음 장: "방 주세요!" 신청을 받는 접수 창구가 있어요.

07. 접수 창구, 서버

7장. 접수 창구, 서버

이 장에서 배울 것: "GPU 방 주세요!" 신청을 받아주는 친절한 접수처가 있어요.

오늘의 목표

1. "백엔드 서버"가 뭔지 알기
2. 방을 신청하면 어떤 순서로 일이 이루어지는지 알기
3. 번호표(세션 id)가 왜 필요한지 알기

도서관 사서 선생님 이야기

학교 도서관에 가면 **사서 선생님**이 계시죠?

책을 빌리고 싶으면 사서 선생님께 말씀드려요.

"선생님, 이 책 빌려주세요."

선생님이 책을 찾아서 빌려주고, **날짜와 이름을 적어두세요.**

나중에 책을 반납하면 선생님이 기록을 지워줘요.

나 → [사서 선생님] → 책장에서 책 꺼내줌
↑
신청을 받고 기록하는 분

우리 프로그램에도 이런 **접수 창구** 역할을 하는 부분이 있어요.

이걸 **백엔드 서버**(Backend Server)라고 불러요.

어려운 이름이지만 뜻은 간단해요.

"뒤에서(백) 일을 처리해 주는(엔드) 컴퓨터 프로그램" 이에요.

우리 프로그램에서는 **FastAPI**(패스트에이피아이)라는 도구로 만들었어요.

FastAPI는 레고처럼 서버를 빠르게 만들 수 있게 해주는 도구예요.

신청서를 받아서 방을 만들어 줘요

누군가가 "GPU 방 주세요!"라고 신청하면,
백엔드 서버는 이렇게 일해요.

순서 1. 신청서를 받아요
"ratio 0.4짜리 방 하나 주세요!"

순서 2. Docker(방 만들기 도구)에게 시켜요
"Docker야, ratio 0.4짜리 방 만들어!"

순서 3. Docker가 방을 만들어요
(컨테이너 방이 생기고, libfgpu.so도 넣어줌)

순서 4. 번호표를 줘요
"방이 생겼어요! 번호는 'abc-123'이에요."

그림으로 보면:

```
graph TD; A[신청하는 사람] --> B["[백엔드 서버] ← 접수 창구"]; B --> C["[Docker] ← 방 만들기 도구"]; C --> D["[컨테이너 방] ← 새로 생긴 방  
(libfgpu.so + quota 들어있음)"]
```

번호표 — 세션 id

방을 만들면 서버가 **번호표**를 줘요.
이 번호표를 **세션 id**(세션 아이디)라고 해요.

왜 번호표가 필요할까요?

식당에서 웨이팅 번호표를 받아본 적 있나요?
번호표가 있어야 "제 테이블 어디예요?"라고 물어볼 수 있잖아요.
GPU 방도 마찬가지예요.
번호표(세션 id)가 있어야 나중에 "내 방 어떻게 됐어?"라고 확인하거나,
"내 방 없애줘!"라고 말할 수 있어요.

번호표 예시:
세션 id = "a3f7bc29-1234-..."

- 이 번호로 할 수 있는 것:
- "이 방 지금 어떤 상태야?" (확인)
 - "이 방에서 뭐가 나왔어?" (로그 보기)
 - "이 방 멈춰줘!" (중지)
 - "이 방 없애줘!" (삭제)

웹 화면에서도 볼 수 있어요

서버가 잘 동작하면 **웹 화면**에서도 방을 신청하고 볼 수 있어요.
인터넷 주소창에 `http://localhost:8000/` 을 치면
방 목록이 짹 나오고, 새 방을 신청하는 버튼도 있어요.

웹 화면 (브라우저)

GPU 방 목록

방번호: a3f7... | 상태: 실행중

[새 방 신청하기 버튼]

방이 너무 많으면 미리 막아줘요

서버는 똑똑해서, **비율 합이 1.0을 넘으면 미리 거절**해요.
(5장에서 배운 것 기억나죠? $0.4 + 0.6 = 1.0$ 이면 딱 맞는 것)

이미 0.7짜리 방이 있는데 0.6짜리를 또 달라고 하면:
 $0.7 + 0.6 = 1.3 \rightarrow 1.0$ 을 넘어!
이럴 때 서버가 "미안, 지금은 공간이 없어요!"라고 거절해요.

이 기능을 **입장 조절**이라고 해요.
5장에서 잠깐 나온 그 이야기예요!
학교 행사에서 "정원이 꽉 찼어요, 못 들어와요!"와 같아요.

한 줄 정리

백엔드 서버는 "GPU 방 주세요!" 신청을 받아서 Docker에게 방을 만들게 하고, 번호표(세션 id)를 나눠줘요.

스스로 물어보기

Q1. 백엔드 서버를 도서관에 비유하면 누구예요?

👉 사서 선생님이에요. 신청을 받고 책(방)을 찾아줘요.

Q2. 세션 id는 왜 필요해요?

👉 내 방이 어떤 방인지 나중에 확인하거나 없애달라고 할 때 쓰려고요. 번호표예요.

Q3. 비율 합이 1.0을 넘으면 어떻게 돼요?

👉 서버가 미리 거절해요. "공간이 없어요!"

➡ 다음 장: 지금까지 배운 부품들을 한 장에 모아서 큰 그림으로 봐요.

08. 전체 설계도

8장. 전체 설계도

이 장에서 배울 것: 지금까지 배운 부품들이 어떻게 하나의 서비스가 되는지 큰 그림으로 봐요.

오늘의 목표

1. 모든 부품의 이름과 역할을 한눈에 보기
2. 부품들이 어떻게 연결되는지 알기
3. 전체가 어떤 흐름으로 동작하는지 알기

지금까지 만난 부품들

이 책에서 하나씩 만났던 부품들을 기억하나요?

- GPU: 계산을 아주 빨리 하는 특별한 두뇌
- `libfgpu.so`: 주문을 가로채서 검사하는 도구
- 컨테이너: 각자의 칸막이 방
- Docker: 방을 만들어 주는 도구
- 백엔드 서버: 신청을 받는 접수 창구
- 기록 공책(DB): 누가 어떤 방을 받았는지 사물함처럼 적어두는 공책
- 웹 화면: 사람이 보고 클릭하는 화면

이 모든 것이 연결되어 하나의 서비스가 돼요.

지금부터 그 **전체 설계도**를 볼 거예요.

큰 그림 — 전체 설계도

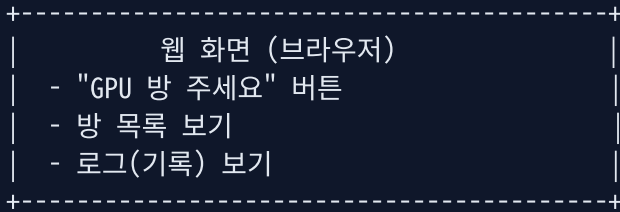
손가락으로 위에서 아래로 따라가 보세요.

사용하는 사람

|

| (버튼 클릭, 신청서 작성)

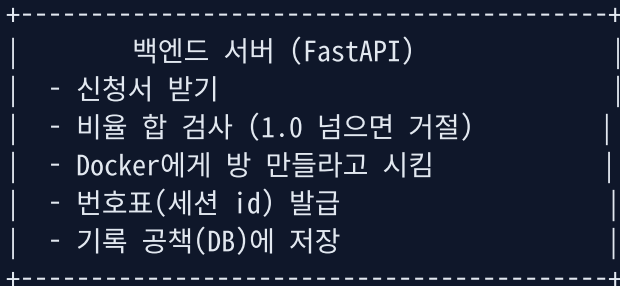
v



|

| (신청 전달)

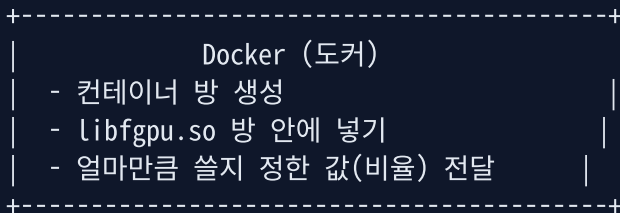
v



|

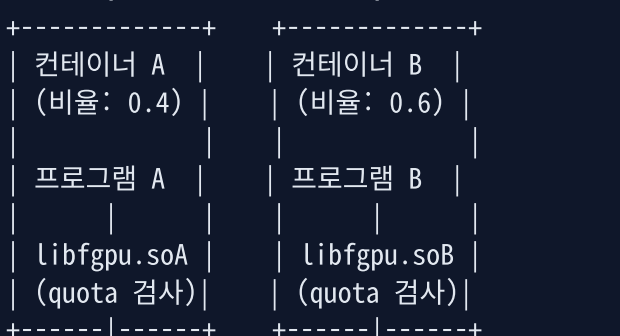
| (방 만들기 명령)

v



|

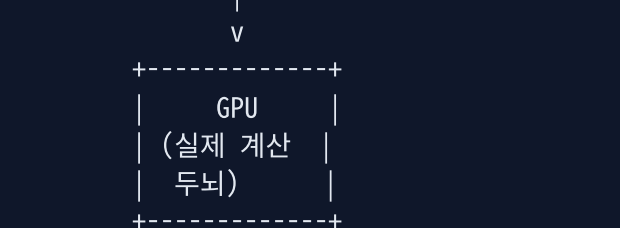
v



|

|

v



각 부품의 역할 한 줄씩

부품 이름	역할 한 줄
웹 화면	사람이 보고 클릭하는 화면
백엔드 서버	신청받고 기록하고 지시하는 접수 창구
기록 공책(DB)	누가 어떤 방을 받았는지 적어두는 사물함 같은 공책
Docker	컨테이너 방을 만들어 주는 도구
컨테이너	각자의 칸막이 방
libfgpu.so	방 안에서 GPU 주문을 가로채서 검사하는 도구
GPU	실제 계산을 엄청 빨리 하는 두뇌

부품들 사이에 무슨 말을 주고받아요?

화살표마다 주고받는 내용이 달라요.

웹 화면 -> 서버:

"비율 0.4짜리 방 만들어 주세요!"

서버 -> Docker:

"비율 0.4인 컨테이너 하나 만들어. libfgpu.so도 넣어."

Docker -> 컨테이너:

방 생성! (libfgpu.so + "이 방은 0.4만큼만 써"라는 메모 붙이기)

컨테이너 안 프로그램 -> libfgpu.so:

"cudaMalloc - GPU야, 3GB 줘!"

libfgpu.so -> 프로그램 (검사 후):

"ALLOW" 또는 "DENY"

libfgpu.so -> GPU (통과한 경우):

진짜 cudaMalloc 요청 전달

서버 -> 웹 화면:

"번호표는 abc-123이에요. 상태: 실행중"

두 겹의 방어막

이 설계에는 **두 겹의 검사**가 있어요.

막는 **때**가 달라서 둘 다 필요해요.

1겹 - 서버에서 미리 막기:

방을 만들기 **전에** 서버가 비율 합이 1.0을 넘으면 미리 거절해요.
(7장에서 배운 입장 조절이에요.)

2겹 - 방 안에서 막기:

방 안에서 프로그램이 실제로 메모리를 달라고 할 **때**,
`libfgpu.so`가 quota를 넘으면 그 자리에서 거절해요.
(4장에서 배운 DENY예요.)

첫 번째 방어막:
서버 접속 창구

"방 만들기 전에
비율 합이 너무 커!"

|

↓

거절
(방 자체를
안 만들어요)

두 번째 방어막:
방 안의 libfgpu.so

"방 안에서 실제로
메모리 달라고 할 때
내 몫을 넘어!"

|

↓

거절 (DENY)

이렇게 두 겹으로 막아서 더 안전해요.

한 줄 정리

사람 -> 웹 화면 -> 서버 -> Docker -> 컨테이너(libfgpu.so 포함) -> GPU.

이 흐름이 **GpuCluster의 전체 설계도**예요.

? 스스로 물어보기

Q1. "방 만들어 줘!"라고 시키는 부품은 누구예요?

👉 백엔드 서버예요. 신청을 받으면 Docker에게 방을 만들라고 시켜요.

Q2. quota 검사는 어디서 해요?

👉 방 안의 `libfgpu.so` 가 해요. 프로그램이 GPU에 주문할 때 가로채서 검사해요.

Q3. 두 겹 방어막이 뭐예요?

👉 서버에서 미리 막기(비율 합 검사) + 방 안에서 막기(libfgpu.so quota 검사)예요. 막는 때가 달라요.

➡ 다음 장: 신청 버튼을 누를 때부터 결과가 나올 때까지, 한 번의 신청이 어떻게 흘러가는지 따라가 봐요.

09. 신청 한 건이 흘러가는 길

9장. 신청 한 건이 흘러가는 길 🚀

이 장에서 배울 것: 버튼을 누르는 순간부터 GPU가 일을 끝낼 때까지, 한 번의 신청이 어떻게 여행하는지 따라가 봐요.

🎯 오늘의 목표

1. 신청 하나가 어떤 순서로 흘러가는지 알기
2. 각 부품들이 서로 무엇을 주고받는지 알기
3. 통과되거나 거절될 때 어떤 일이 일어나는지 알기

편지 배달 이야기

편지를 보내는 과정을 생각해 봐요.

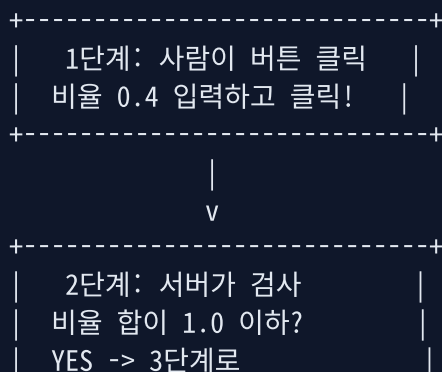
편지를 쓰고 -> 우체통에 넣으면 -> 집배원이 가져가고 -> 분류 센터를 거쳐 -> 받는 사람 집에 도착해요.

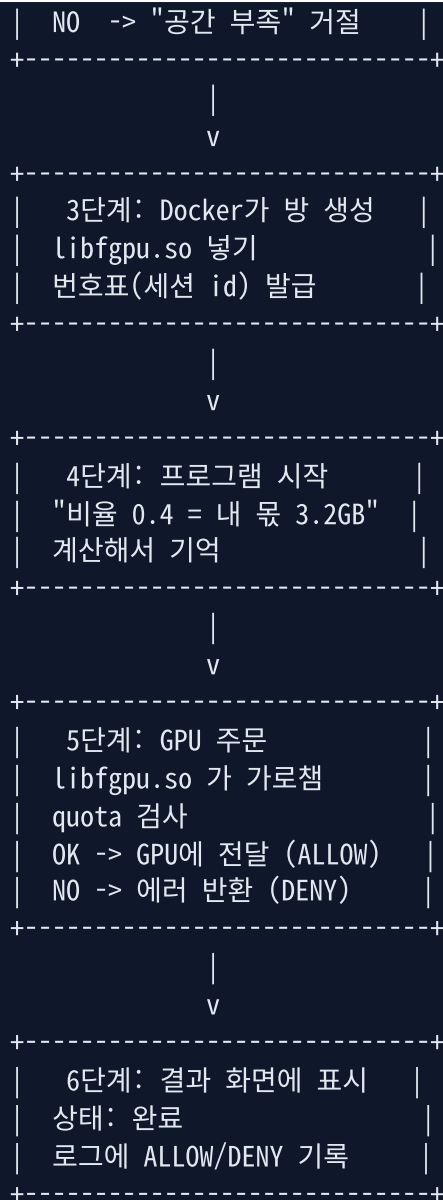
여러 단계를 거치지만, 각 단계에서 정해진 일을 해요.

GPU 신청도 똑같이 **여러 단계**를 거쳐요.

한 번 따라가 봐요!

전체 흐름도 — 위에서 아래로 따라가요





단계별로 자세히 보기

1단계: 사람이 버튼을 클릭해요

웹 화면에서 신청서를 써요.

- "비율은 0.4로 해주세요"

- "이 프로그램을 실행해 주세요"

그리고 버튼을 클릭해요.

웹 화면

| 비율: [0.4] |

```
| 프로그램: [ AI 프로그램 ] |  
|                               |  
|   [ 신청 버튼! ]   |  
+-----+  
      클릭! |  
            v
```

2단계: 서버가 신청을 받고 검사해요

서버가 신청서를 받아요.

먼저 **비율 합 검사**를 해요.

```
서버 (접수 창구):  
"지금 0.7짜리 방이 이미 있는데, 0.4를 또 달라고?  
0.7 + 0.4 = 1.1 -> 1.0을 넘어! 거절!"
```

또는:

```
"지금 아무것도 없네. 0.4는 1.0 이하. 통과!"
```

통과하면 3단계로 가요.

거절되면 웹 화면에 "공간 부족" 메시지가 나와요.

3단계: Docker가 방을 만들어요

서버가 Docker에게 명령해요.

Docker가 새 컨테이너 방을 만들어요.

방을 만들 때 이것들을 넣어줘요.

- **libfgpu.so** (가로채기 도구)
- "이 방은 0.4만큼만 써"라는 메모 (4장에서 배운 먼저 끼어들기 목록에 같이 적어줘요)
- 실행할 프로그램

Docker가 방을 만드는 순서:

1. 빈 방 하나 준비
2. libfgpu.so 넣기
3. "이 방은 0.4만큼만 써"라고 방 안에 메모 붙이기
4. 프로그램 준비
5. 방 문 열기 (프로그램 시작!)

방이 생기면 서버는 번호표(세션 id)를 웹 화면에 보내줘요.

4단계: 방 안에서 프로그램이 시작돼요

방 문이 열리면 프로그램이 시작돼요.

`libfgpu.so` 가 "이 방은 0.4만큼만 써" 메모를 읽어요.

그리고 "GPU 전체의 40%가 내 몫이구나!"를 계산해서 기억해 뒀어요.

```
libfgpu.so 준비:  
"메모 확인: 비율 0.4  
GPU 전체 8GB x 0.4 = 3.2GB.  
나는 3.2GB까지만 허용할게."
```

5단계: GPU에 주문이 들어가요

프로그램이 일을 시작하면 GPU에 메모리를 달라고 해요.

바로 이때 `libfgpu.so` 가 먼저 받아요.

(04장에서 배운 그 검사예요!)

```
프로그램 -> "cudaMalloc: 1GB 줘!"  
      |  
      v  
libfgpu.so 가 받음  
      |  
      v  
1단계: 지금 쓰는 양 = 0  
2단계: 이번에 달라는 양 = 1GB  
3단계: 0 + 1GB = 1GB <= 3.2GB (내 몫)  
-> ALLOW! 진짜 GPU에 전달  
      |  
      v  
GPU 가 1GB 줌  
(방금 1GB를 받았으니 쓴 양이 1GB가 된 거예요!)
```

나중에 프로그램이 더 요청하면:

```
프로그램 -> "cudaMalloc: 3GB 더 줘!"  
      |  
      v  
libfgpu.so 가 받음  
      |
```

v

1단계: 지금 쓰는 양 = 1GB
2단계: 이번에 달라는 양 = 3GB
3단계: 1GB + 3GB = 4GB > 3.2GB (내 몫 초과!)
→ DENY! 에러 반환

|

v

프로그램에 "메모리 부족" 에러
(GPU 는 아무것도 모름)

6단계: 결과가 화면에 나와요

프로그램이 끝나면 상태가 바뀌어요.

웹 화면에서 새로고침하면 "완료" 상태가 돼요.

로그 버튼을 누르면 **libfgpu.so** 가 남긴 기록이 나와요.

웹 화면 - 로그 창:

```
[fgpu] ALLOW cudaMalloc size=1GB used=1/3.2GB  
[fgpu] DENY  cudaMalloc size=3GB used=1/3.2GB  
[fgpu] FREE  size=1GB          used=0/3.2GB
```

전체 흐름 한 줄씩 요약

1. 사람이 버튼 클릭 (비율 0.4 입력)
2. 서버가 신청 받고 비율 합 검사 (초과면 "공간 부족" 거절)
3. Docker가 방 만들고 번호표 발급 (libfgpu.so + 비율 메모 포함)
4. 방 안에서 프로그램 시작 (libfgpu.so가 "내 몫 3.2GB" 계산)
5. GPU 주문 → libfgpu.so 가 가로채서 quota 검사 (ALLOW or DENY)
6. 결과 화면에 표시 (상태 "완료", 로그에 ALLOW/DENY 기록)

한 줄 정리

버튼 하나를 누르면 여섯 단계를 거쳐요.

각 단계마다 다른 부품이 자기 일을 해요.

? 스스로 물어보기

Q1. 2단계(서버 검사)에서 거절되는 건 어떤 상황이에요?

👉 이미 있는 방들의 비율 합 + 새 요청 비율 > 1.0일 때예요.

Q2. `libfgpu.so`가 DENY를 보낼 때 GPU는 알아요?

👉 몰라요. `libfgpu.so`가 GPU에 주문을 보내기 전에 막으니까요.

Q3. 로그에서 ALLOW/DENY를 어디서 볼 수 있어요?

👉 웹 화면의 로그 창에서 볼 수 있어요.

➡ 다음 장: 두 명이 같이 쓰면 더 빠를까요? 실제로 실험해 봤어요!

10. 같이 쓰면 빨라질까? 우리 실험

10장. 같이 쓰면 빨라질까? 우리 실험

이 장에서 배울 것: 두 명이 GPU를 같이 쓰면 어떻게 되는지 실제로 실험해 봤어요. 결과가 솔직하고 재미있어요!

오늘의 목표

1. GPU를 같이 쓸 때 어떤 일이 생기는지 알기
2. 메모리 칸은 잘 나뉘지만 "일꾼"은 함께 쓴다는 걸 알기
3. 번갈아 쓰기(throttle)가 무엇인지 알기

실험 이야기

우리가 만든 프로그램으로 실제로 실험했어요.

실험에는 **작은 챗봇 AI**를 썼어요.

이 챗봇은 질문을 받으면 글자 조각(토큰)을 한 개씩 차례로 만들어서 대답을 써요.

(토큰은 글자나 짧은 낱말 조각 정도예요. "안녕하세요"가 몇 개의 토큰으로 나뉜다고 생각하면 돼요.)

이 챗봇이 **1초에 몇 개의 글자 조각을 만드는지**가 빠르기예요.

많이 만들수록 대답이 빨리 나와요.

우리는 이 챗봇을 이렇게 두 가지 상황에서 실험했어요.

친구 A와 B가 같은 GPU를 나눠 써요.

- A는 비율 0.4 (전체의 40%)
- B는 비율 0.6 (전체의 60%)

결과가 어떻게 됐을까요?

메모리(책상 자리)는 잘 나뉘어요

먼저 좋은 소식이에요.

메모리 나누기는 잘 됐어요.

A가 자기 몫(40%)을 넘게 달라고 하면 -> DENY!

B가 자기 몫(60%) 안에서 달라고 하면 -> ALLOW!

둘이 같은 GPU를 쓰지만, 각자 자기 몫 안에서만 써요.

서로 방해하지 않고, 서로 칸을 빼앗지 않아요.

GPU 메모리 (8GB)

```
+-----+
|  A 칸 (3.2GB) | B 칸 (4.8GB)  |
|  A만 사용 가능 | B만 사용 가능  |
+-----+
"책상 자리는 깔끔하게 나뉨!"
```

이것만으로도 아주 유용해요.

A의 실수가 B에게 영향을 안 주고, B의 실수가 A에게 영향을 안 줘요.

그런데 계산 일꾼은 함께 써요

이제 솔직한 이야기예요.

GPU 안에는 **계산을 하는 일꾼들**이 있어요.

(우리는 그냥 "일꾼들"이라고 부를게요.)

이 일꾼들은 **칸막이로 나눌 수 없어요.**

A의 일과 B의 일이 같은 일꾼들을 두고 경쟁해요.

```
A의 일 --+
          +--> [GPU 일꾼들] <-- 둘이 같이 씬!
B의 일 --+
```

그래서 둘이 동시에 열심히 일하면 둘 다 느려져요.

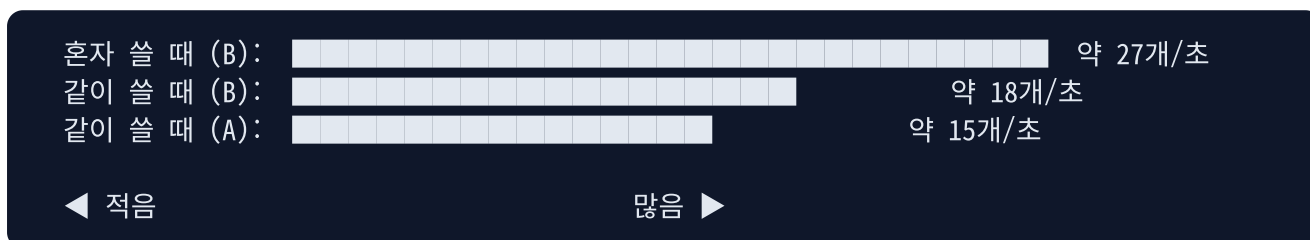
진짜 측정한 숫자로 보여 드릴게요.

진짜 측정 결과 (1초에 글자조각 몇 개?)

(A는 비율 0.4, B는 비율 0.6으로 맞추고, 한 번 대답할 때 토큰 128개를 만들게 해서 잰어요.)

상황	1초에 글자조각	글자조각 128개 걸린 시간
B가 혼자 GPU를 쓸 때	약 27개	약 4.7초
B가 A와 같이 쓸 때	약 18개	약 7.6초
A가 B와 같이 쓸 때	약 15개	약 11초

눈으로 한눈에 봐요.



B는 혼자 쓸 때 1초에 27조각을 척척 만들었어요.

그런데 A와 같이 쓰니까 18조각으로 떨어졌어요.

A는 15조각밖에 못 만들었어요.

여기서 중요한 점! 비율 0.4 / 0.6은 **책상(메모리)을 나누는 약속**이지, **계산 일꾼을 나누는 게 아니에요**. 일꾼은 A와 B가 똑같이 두고 경쟁해요. 그래서 A가 40%만 받았어도, 둘 다 열심히 일하면 일꾼을 나눠 갖느라 B도 많이 느려지는 거예요.

9장까지 말로만 했던 걸 이제 진짜 숫자로 확인했어요!

운동장 비유로 생각해요.

체육 시간에 1반이 축구를 하는데 2반도 같은 운동장에서 달리기를 해요.

운동장을 나눠 쓰긴 하지만, 좁아지면 서로 느려질 수 있잖아요.

한 명이 쉬엄쉬엄 쓰면 다른 친구가 빨라져요

재미있는 실험 결과도 있어요.

A가 쉬엄쉬엄 하고 있으면 (일을 조금씩 할 때),

일꾼들이 B의 일을 더 많이 도울 수 있어요.

그럼 B는 더 빨리 끝나요.

A가 쉬는 중...

일꾼들: "A 일이 없으니까 B 일 더 빨리 하자!"
-> B가 빨라짐!

이건 비유하면, 쉬는 시간에 운동장이 텅 비면
나 혼자 뛸 때 훨씬 넓게 쓸 수 있잖아요.

번갈아 쓰기 — Throttle (쓰로틀)

그렇다면 "A가 너무 열심히 할 때 조금 쉬게 하면 어떨까?" 하는 아이디어가 생겨요.

이걸 **throttle**(쓰로틀)이라고 해요.

쉬운 말로 하면 "**번갈아 쓰기**" 또는 "**속도 조절**"이에요.

예를 들어 A에게 비율 0.4(40%)를 정해 주면,

A는 정한 시간의 40%만 일하고 나머지는 쉬어요.

혼자라면 1초에 약 27개를 만들 텐데, 0.4로 속도를 조절하면 약 10~11개 정도로 줄어드는 셈이
예요.

"정한 비율만큼 느려진다"는 걸 숫자로 확인할 수 있어요.

그네 순서 정하기 비유

놀이터 그네가 하나밖에 없어요.

A와 B가 둘 다 그네를 타고 싶어요.

공평하게 하는 방법:

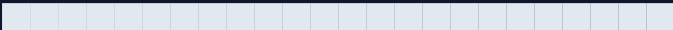
A가 1분 타면 -> 잠깐 기다려.

B가 1분 타면 -> 잠깐 기다려.

번갈아 타요!

쓰로틀이 **없을 때**는 B가 계속 쉬지 않고 그네를 탈 수 있어요.

시간 ->

B:  (계속 탐, 쉬지 않음)

쓰로틀이 **있을 때**는 A와 B 각자 정한 비율만큼만 일하고 쉬어요.

쓰로틀은 각 방에 따로 거는 거예요.

A는 40%만 일하고 쉬고, B는 60%만 일하고 쉬어요. 각자 정한 만큼만 일하는 거예요.

딱 맞춰 번갈아 도는 건 아니예요. 둘 다 동시에 쉴 때도 있어서 그땐 GPU가 잠깐 놀기도 해요.

시간 ->

A:  (40%만 일하고 나머지는 쉼)
B:  (60%만 일하고 나머지는 쉼)
(두 줄이 따로 움직여요 — 칼같이 엇갈리진 않아요)

우리 프로그램의 번갈아 쓰기는 이렇게 해요

우리 가로채기 도구(libfgpu.so)는 GPU에 일을 시킬 때도 살짝 가로채요.

(GPU에 "계산해!"라고 시키는 것도 `cudaLaunchKernel` 이라는 주문이거든요.)

이 주문을 가로채서, **정해진 비율만큼 시간이 지나면 잠깐 기다리게** 해요.

1초를 1000조각으로 나눈 것 중 하나가 **1ms(밀리초)**예요.

100ms면 1초의 10분의 1, 아주 짧은 시간이에요.

우리는 100ms 창 안에서 정해진 비율만큼만 허용해요.

비율 0.4 일 때:

100ms 창에서 40ms만 일하고, 나머지 60ms는 잠깐 기다림.

```
+-----+
|           100ms 한 칸           |
| [일함: 40ms]   [기다림: 60ms]   |
+-----+
```

-> 다음 100ms 창에서 또 반복

중요한 점: **주문 자체를 버리지는 않아요.**

잠깐 기다렸다가 반드시 GPU에 전달해요.

기다리는 것뿐, 없애는 건 없어요!

실험 결과 요약

상황	결과
B 혼자 쓸 때	약 27 글자조각/초 (빠름)
B가 A와 같이 쓸 때	약 18 글자조각/초 (느려짐)
A와 B 둘 다 열심히	둘 다 느려져요 (계산 일꾼을 나눠 써서)
A가 쉬엄쉬엄	B가 더 빨라져요
번갈아 쓰기(throttle) 켜	A의 속도가 정한 비율만큼 줄어듭니다
메모리 나누기	잘 됐어요! 서로 방해 없어요

한 줄 정리

메모리 책상 자리는 잘 나뉘지만,
계산 일꾼은 함께 써요.
번갈아 쓰기(throttle)로 비율만큼 쉬게 하면
더 공평하게 나눌 수 있어요.

스스로 물어보기

Q1. GPU 메모리 나누기는 잘 됐나요?

👉 네! A 몫과 B 몫이 따로따로 지켜져요.

Q2. A가 열심히 일할 때 B가 느려질 수 있는 이유는?

👉 계산 일꾼들을 A와 B가 같이 쓰기 때문이에요. 칸막이가 없어요.

Q3. throttle(번갈아 쓰기)이 주문을 없애나요?

👉 아니요! 잠깐 기다리게 할 뿐, 주문 자체는 반드시 실행돼요.

➡ 다음 장: 이 프로그램이 아직 못 하는 것들을 솔직하게 알려줄게요.

11. 아직 못 하는 것 (정직한 한계)

11장. 아직 못 하는 것 (정직한 한계) 🤔

이 장에서 배울 것: 우리 프로그램이 못 하는 것들을 솔직하게 알아봐요. 못 한다고 나쁜 게 아니에요!

🎯 오늘의 목표

1. 우리 프로그램이 잘 하는 것과 못 하는 것 구분하기
2. 왜 못 하는지 이해하기
3. 그래도 충분히 쓸모 있다는 걸 알기

솔직하게 이야기해요

어려워 보이죠? 사실 별거 아니에요.

우리 프로그램이 못 하는 것들이 있어요.

그걸 숨기는 게 아니라 **솔직하게** 말하는 게 좋아요.

과학에서도 "이건 됐고, 이건 아직 안 됐어"라고 말하는 게 더 믿음이 가잖아요.

우리도 그렇게 할 거예요.

못 하는 것 1: 계산 일꾼을 진짜로 칸막이 치지 못해요

10장에서 배운 것 기억하죠?

GPU 안에는 **계산 일꾼들**이 있어요.

우리는 **메모리(책상 자리)** 는 잘 나눠줄 수 있어요.

하지만 **계산 일꾼들** 에 칸막이를 치진 못해요.

우리가 할 수 있는 것:

메모리: A 칸 | B 칸 ← 잘 됨! |

우리가 못 하는 것:

계산 일꾼: 모두 함께 쓰는 상태
← 칸막이 없음

그래서 A와 B가 동시에 열심히 일하면
계산 일꾼들을 두고 경쟁하게 돼요.
둘 다 조금 느려질 수 있어요.

왜 못 하나요?

계산 일꾼을 칸막이로 나누는 건 **특별한 하드웨어 기능**(MIG라고 해요)이 있어야 해요.
그런데 우리가 쓰는 GPU(RTX 4060, 4070 같은 일반 게임용 GPU)는 그 기능이 없어요.
그래서 소프트웨어로는 아무리 노력해도 못 해요.

(번갈아 쓰기 throttle로 어느 정도 도움이 되지만, 진짜 칸막이는 아니에요.)

못 하는 것 2: 규칙을 안 지키기로 마음먹은 프로그램은 못 막아요

우리 가로채기 도구(libfgpu.so)는 **LD_PRELOAD**를 이용해요.
(4장에서 배운 "가짜 점원을 먼저 세우는 방법"이요.)

그런데 만약 어떤 프로그램이 **가짜 점원을 무시하고** 직접 주방에 들어가려 하면?
우리는 막을 수 없어요.

예를 들어:

- 일반적인 AI 프로그램 → 우리 도구가 잘 가로채요.
- 규칙을 무시하고 뒷문으로 몰래 들어가는 프로그램 → 우리 도구를 피해 갈 수 있어요.

일반 프로그램:

프로그램 → [가짜 점원 libfgpu.so] → GPU
(잘 가로챔!)

규칙 안 지키는 프로그램:

프로그램 → GPU
(가짜 점원 무시!)

그래서 이 프로그램은 "**서로 믿고 협력한다**"는 가정 아래서 동작해요.
친한 친구들끼리, 또는 같은 학교 친구들끼리 나눠 쓰는 상황이에요.
일부러 규칙을 깨려는 나쁜 프로그램은 막지 못해요.

못 하는 것 3: 완벽한 하드웨어 격리는 불가능해요

아주 비싼 전문가용 GPU(A100, H100 같은 것)는 **MIG**(엠아이지)라는 기능이 있어요.
MIG가 있으면 GPU를 **하드웨어 수준에서** 완전히 나눌 수 있어요.
계산 일꾼도, 메모리도, 완벽하게 따로따로.

우리가 쓰는 GPU에는 그 기능이 없어요.
그래서 우리는 소프트웨어로 최대한 흉내를 낼 뿐이에요.

완벽한 하드웨어 격리 (MIG):

A 전용 GPU 조각	B 전용 조각
(메모리+일꾼 완전 분리)	

→ 아주 비싼 GPU에서만 가능

우리 방식 (소프트웨어):

메모리: A 칸 / B 칸 (잘 됨)	
일꾼: 함께 씬 (분리 불가)	

→ 일반 게임용 GPU에서도 됨!

그래도 충분히 쓸모 있어요!

못 하는 것도 있지만, 잘 하는 것도 분명히 있어요.

우리가 잘 하는 것:

- ✓ 메모리(책상 자리) 나누기
- ✓ 각자 몫을 넘으면 정확히 거절하기
- ✓ 여러 명이 동시에 방을 쓸 수 있게 하기
- ✓ 방 신청, 확인, 삭제를 쉽게 하기
- ✓ 번갈아 쓰기로 어느 정도 공평하게 하기

일반 게임용 GPU(RTX 4060, 4070)로도 여러 명이 나눠 쓸 수 있어요.
이건 아무것도 없을 때보다 훨씬 낫죠!

학교 친구들끼리 GPU를 나눠 쓰고 싶을 때,
또는 회사 동료들끼리 같은 GPU를 쓰고 싶을 때,
이 프로그램이 도움이 돼요.

한 줄 정리

메모리 나누기는 잘 돼요.
하지만 계산 일꾼 칸막이, 나쁜 프로그램 막기, 완벽한 하드웨어 격리는 못 해요.
그래도 일반 GPU를 나눠 쓰는 데는 충분히 쓸모 있어요!

? 스스로 물어보기

Q1. 우리 프로그램이 잘 나눠주는 건 뭐예요?

👉 메모리(책상 자리)예요. 각자 몫을 잘 지켜줘요.

Q2. 계산 일꾼을 진짜로 나눠주려면 뭐가 필요해요?

👉 MIG라는 특별한 하드웨어 기능이 필요해요. 일반 GPU에는 없어요.

Q3. 이 프로그램은 어떤 상황에 쓸모 있어요?

👉 친구들이나 동료들끼리 일반 GPU를 나눠 쓸 때요. 서로 믿는 상황에서요.

이 책을 다 읽은 여러분에게

여기까지 읽었다면 정말 대단해요!

처음에는 GPU라는 말도 낯설었을 거예요.
LD_PRELOAD, cudaMalloc, Docker, FastAPI...
이 모든 어려운 이름들이 이제는 조금 친숙하죠?

이 프로그램은 비싼 GPU를 여러 명이 나눠 쓸 수 있게 만든 거예요.
완벽하지 않아도, 충분히 유용해요.

앞으로 더 좋은 기술이 나오고, 이 프로그램도 더 발전할 거예요.
여러분도 나중에 이런 것을 만들 수 있어요. 어렵지 않아요.

다 같이 고생했어요! 수고했어요. 🎉

🏠 처음으로: [아주 쉬운 GpuCluster 교과서](#)